1. Given a processor implemented by a single-cycle control and datapath shown in Figure 1. Assume that the functional blocks adopted to implement the datapath have the following latencies in Table 1, and we need to design a processor that can support the MIPS instructions listed in Table 2:

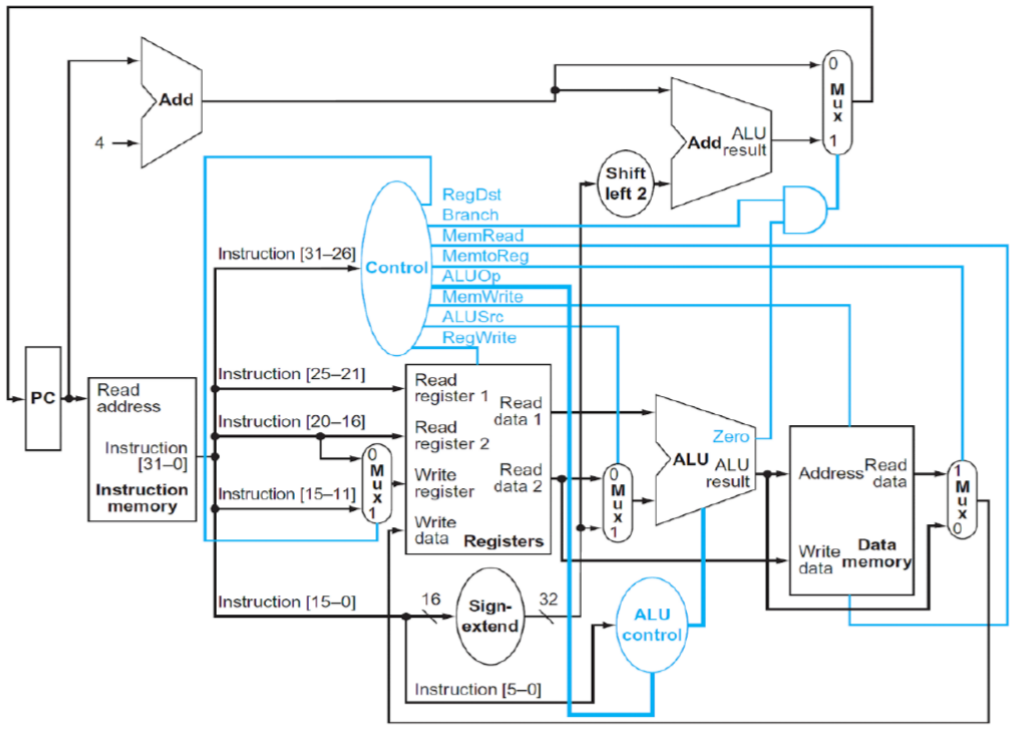


Figure 1. The single-cycle datapath with the control unit

Table 1. Latencies of the functional units on the datapath

PC's Clk.-to-Q	I-Mem.	Add	Mux	ALU	Ctrl. Unit	ALU-Ctrl.	Regs. Access	Data-Mem.	Sign-extend	AND-Gate	Shift-left-2
0ps	200ps	70ps	40ps	120ps	200ps	60ps	100ps	200ps	20ps	50ps	20ps

Table 2. MIPS Instruction Opcode & Funct fields

Instr.	Opcode	Funct
add	0_{10}	32_{10}
sub	0_{10}	34_{10}
beq	4_{10}	
lw	35_{10}	
sw	43_{10}	

1.1 If the only thing we need to do in a processor is fetch consecutive instructions (Figure 4.6), what would the cycle time be?

0 + 200ps = 200 ps

ANS: 200ps

1.2 If the thing we need to do in a processor are fetch consecutive instructions (Figure 4.6) and decode the instruction, what would the cycle time be?

PC I-MEM

0 + 200ps = 200ps

A: 200 ps

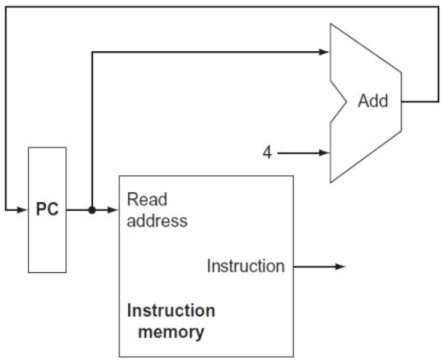


FIGURE 4.6 A portion of the datapath used for fetching instructions and incrementing the program counter. The fetched instruction is used by other parts of the datapath.

1.3. If this processor only need to support **sw** and **beq** and **sub** instructions, what is the **clock cycle time**? (Hint: You should analyze the critical path of the required instructions)

sw : 200 (I-MEM) + 200 (Control) + 60 (ALU Control) + 120 (ALU) + 200 (Data MEM) = 780 ps #

beq : 200 (I-MEM) + 200 (Control) + 60 (ALU Control) + 120 (ALU) + 50 (AND) + 40 (MUX) = 670 ps

sub : 200 (I-MEM) + 200 (Control) + 60 (ALU Control) + 120 (ALU) + 40 (MUX) = 620ps

ANS: 780 ps #

1.4. What is the **clock cycle time** of this processor that support the instructions listed in Table 2? (Hint: lw 請使用 I-Mem + Ctrl Unit + ALU Ctrl + ALU + Data-Mem + Mux 公式計算)

lw : 200 (I-MEM) + 200 (Control) + 60 (ALU Control) + 120 (ALU) + 200 (Data Mem) + 40 (MUX) = 820 ps

add : 200 (I-MEM) + 200 (Control) + 60 (ALU Control) + 120 (ALU) + 40 (MUX) = 620ps

ANS: 820ps #

2. In this exercise we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to **3 clock** cycle in which the processor fetches the following **3 instructions word:**

op rs (9) rt (10) rd (11) shamt func

0000 0001 0010 1010 0101 1000 0010 0010

sw $8, 2($9)

beq $8, $($9), 20

Assume that **data memory is all zeros** and **that the processor's registers have the following values** at the beginning of the cycle in which the above instruction word is fetched (Please refer to the MIPS Instruction Opcode & Funct fields listed in ***Table 2***.):

r0	*r1*	*r2*	*r3*	*r4*	*r5*	*r6*	*r8*	*r9*	*r10*	*r11*	*r31*
0	-1	9	-3	-4	10	11	8	5	7	4	-16

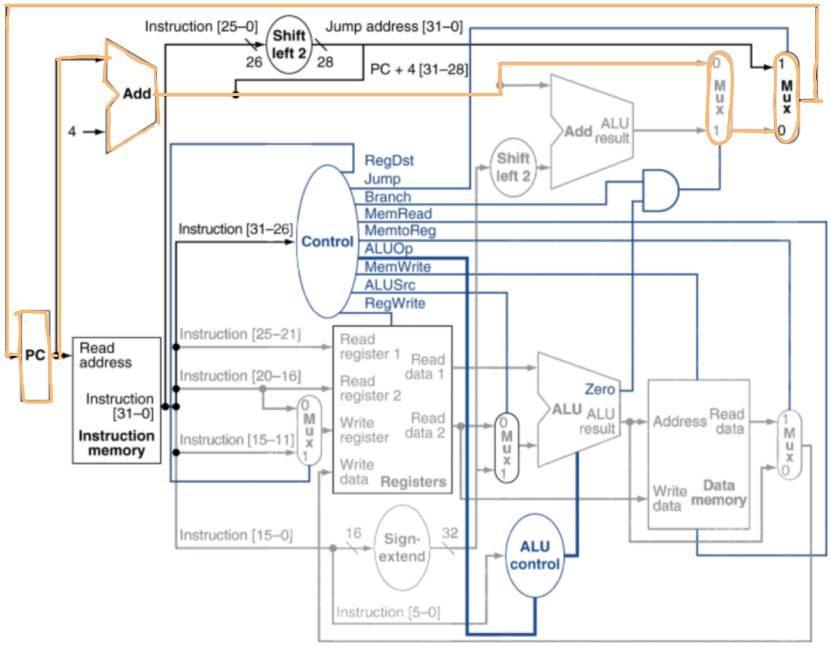


Figure 2.

2.1. What are the **3** outputs of the sign-extend and the jump "Shift left-2" unit (near the top of Figure 4.24) for this instruction word?

Instruction 1 : sub $11, $9, $10 (000000 01001 01010 01011 00000 100010)

Sign Extended = 00000000 00000000 01011000 00100010 #

Shift Left-2 = 0100 10101001 01100000 10 001000 #

2 : sw $8, 2($9) (101011 01001 01000 0000 0000 0000 0010)

Sign Extended = 00000000 00000000 00000000 00000010 #

Shift Left-2 = 01001 01000 0000 0000 0000 0010 000 #

3 : beq $8, $9, 20 (000100 01000 01001 00000000 000 10100)

Sign Extended = 00000000 00000000 00000000 00010100 #

Shift Left-2 = 01000 01001 00000000 000 10100 00 #

2.2. What are the values of the ALU control unit's inputs for **these instructions?**

Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUOp0
R-format ✓	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw ✓	X	1	X	0	0	1	0	0	0
beq ✓	X	0	X	0	0	0	1	0	1

Instruction	$ALU_{OP(1)}$	$ALU_{OP(0)}$	Instructions (6bit)
1	1	0	100010
2	0	0	000010
3	0	1	010100

2.3. What is the new PC address after **these instructions are** executed? **And, highlight** the path through which this value is determined.

ANS: (1) $PC_{new} = PC_{old} + 4$

(2) 答案已標示在 Figure 2

2.4. For each Mux, show **the control signal of Mux and** the values of its data output during the execution of **these instructions and these register values**.

RegDst Mux		ALUsrc Mux		PCsrc Mux		MemtoReg Mux	
1	11	0	7	0	PC+4	0	-2
X	X	1	2	0	PC+4	X	X
X	X	0	5	0	PC+4	X	X

2.5. For the ALU and the two add units, what are their data input values?

ALU	Add	Add (ALU result)
5, 7	PC, 4	PC+4, 00000000 00000001 01100000 10001000 (2)
5, 2	PC, 4	PC+4, 00000000 00000000 00000000 00001000 (2)
8, 5	PC, 4	PC+4, 00000000 00000000 00000000 01010000 (2)

2.6. What are the values of all inputs for the "Registers" unit?

Read Register 1	Read Register 2	Write Register	Write Data	RegWrite
9	10	11	-2	1
9	8	X	X	0
8	9	X	X	0

3. Given a processor implemented by a single-cycle control and datapath shown in follow. Assume that the functional blocks adopted to implement the datapath have the following latencies:

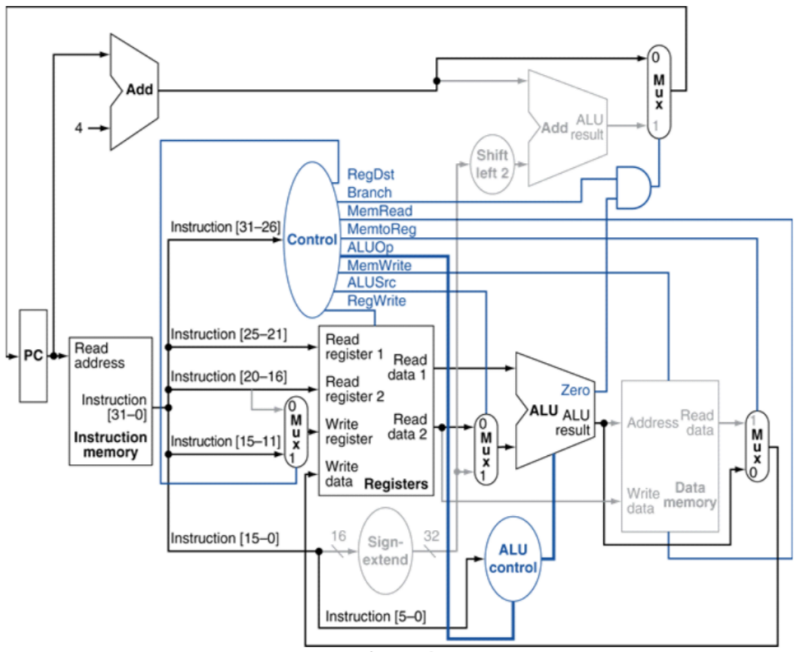


Figure 3.

Inst.-Mem.	Add	Mux	ALU	ALU-Ctrl	Regs. Access	Data-Mem.	Sign-extend	Shift-left-2
500ps	350ps	100ps	250ps	100ps	400ps	600ps	100ps	50ps

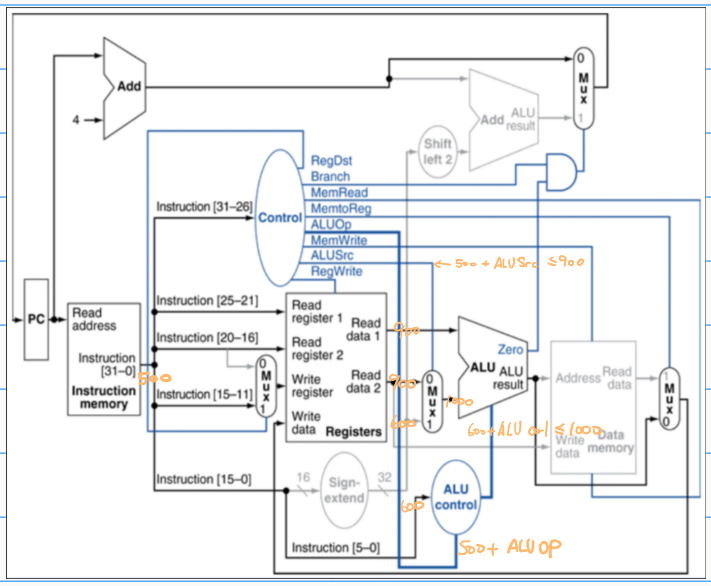
3.1. For add instruction's critical path shown in Figure, **which control signal is the most critical** that is needed to be quickly generated, and **what is the maximum time** that the control unit can generate this signal to avoid lengthening the critical path? (Hint: the critical path of add instruction)

ALU Source MUX Input delay = 500 (INS-MEM) + 400 (REG-FILE) = 900

Control Input delay = 500 (INS-MEM)

900 - 500 = 400

A: (1) ALU Src MUX (2) 400ps



3.2. Assume that the control unit shown in Figure requires the times to generate the individual control signals as the following table. What is the clock cycle time of this processor?

RegDst	Jump	Branch	MemRead	MemtoReg	ALUOp	MemWrite	ALUSrc	RegWrite
1440ps	1000ps	1000ps	600ps	1000ps	500ps	800ps	350ps	1300ps

lw : 500 (I-MEM) + 500 (ALU-OP) + 100 (ALU-Ctrl) + 250 (ALU) + 600 (Data-MEM) + 100 (MUX) = 2050

sw : 500 (I-MEM) + 500 (ALU-OP) + 100 (ALU-Ctrl) + 250 (ALU) + 600 (Data-MEM) = 1950

R-Format : 500 (I-MEM) + 1440 (Reg-Dst) + 100 (MUX) = 2040 A : 2050 ps

3.3 If the only thing we need to do in a processor is fetch consecutive instructions (Figure 4.6), what would the cycle time be?

500 (I-MEM) A:500 ps

3.4 Consider a datapath similar to the one in Figure 4.11, but for a processor that only has one type of instruction: unconditional PC-relative branch. What would the cycle time be for this datapath?

500 (I-MEM) + 100 (Sign-ext) + 50 (SHIFT-2) + 350 (ADD) + 100 (MUX) = 1100 A:1100ps

3.5 Repeat 3.4, but this time we need to support only conditional PC-relative branches.

500 (I-MEM) + 1000 (Branch) + 100 (MUX) = 1600 A:1600 ps

3.6 Which kinds of instructions require the datapath element Shift-left-2 resource? For which kinds of instructions (if any) is this resource on the critical path?

ANS: (1) beq、j 指令

(2) 在 beq 中不是 critical path，因為 Control Branch 的時間遠大於 Sign_ext + Shift_2_left。在 j 指令中即是 critical path，ADD 遠小於 Sign_ext + Shift_2_left。

4. When processor designers consider a possible improvement to the processor data path, the decision usually depends on the cost/performance trade-off. In the following three problems, assume that we are starting with a data path from Figure 4.2, where I-Mem, Add, Mux, ALU, Registers, D-Mem, and Control blocks have latencies of 500ps, 200ps, 50ps, 200ps, 300ps, 450ps, and 100ps, respectively, and costs of 1500, 35, 20, 100, 200, 2000, and 500 (dollars), respectively. Consider the addition of a multiplier to the ALU. This addition will add 350ps to the latency of the ALU and will add a cost of 580 dollars to the ALU. The result will be 10% fewer instructions executed since we will no longer need to emulate the MUL instruction.

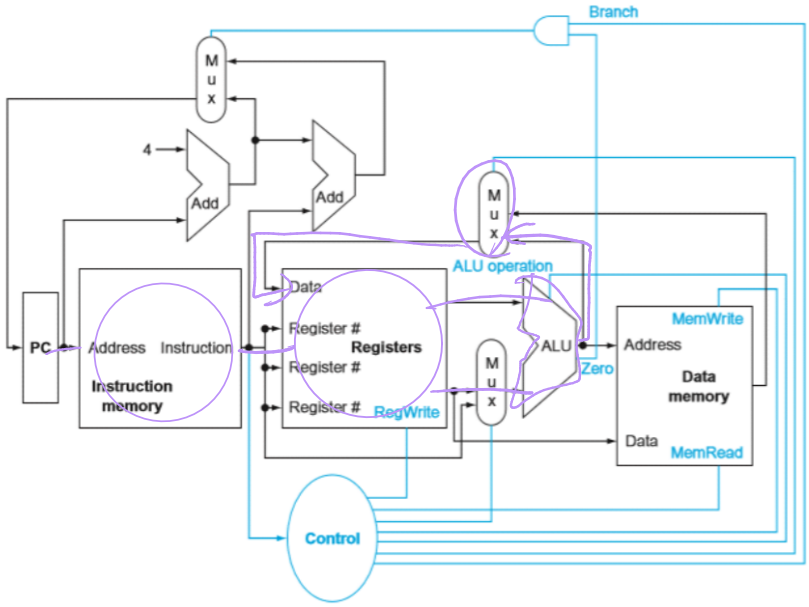


FIGURE 4.2 The basic implementation of the MIPS subset, including the necessary multiplexors and control lines. The top multiplexor ("Mux") controls what value replaces the PC (PC + 4 or the branch destination address); the multiplexor is controlled by the gate that "ANDs" together the Zero output of the ALU and a control signal that indicates that the instruction is a branch. The middle multiplexor, whose output returns to the register file, is used to steer the output of the ALU (in the case of an arithmetic-logical instruction) or the output of the data memory (in the case of a load) for writing into the register file. Finally, the bottommost multiplexor is used to determine whether the second ALU input is from the registers (for an arithmetic-logical instruction or a branch) or from the offset field of the instruction (for a load or store). The added control lines are straightforward and determine the operation performed at the ALU, whether the data memory should read or write, and whether the registers should perform a write operation. The control lines are shown in color to make them easier to see.

	I-Mem	Add	Mux	ALU	ALU With Multiplier	Regs	D-Mem	Control
latency	500ps	200ps	50ps	200ps	550ps	300ps	450ps	100ps
cost	1500	35	20	100	680	200	2000	500

4.1 What is the clock cycle time with and without this improvement?

Without: $\underset{\text{I-MEM}}{500} + \underset{\text{Regs}}{300} + \underset{\text{MUX}}{50} + \underset{\text{ALU}}{200} + \underset{\text{MUX}}{50} + \underset{\text{D-MEM}}{450} = 1550$

With: $\underset{\text{I-MEM}}{500} + \underset{\text{Regs}}{300} + \underset{\text{MUX}}{50} + \underset{\text{ALU-Latency}}{350+200} + \underset{\text{MUX}}{50} + \underset{\text{D-MEM}}{450} = 1900$

(1) Without: 1550 ps

A: (2) With: 1900 ps

4.2 What is the speedup achieved by adding this improvement?

$\frac{1}{0.9} \times \frac{1550}{1900} \approx 0.9064$

A: 0.906

4.3 Compare the cost/performance ratio with and without this improvement.

$Cost_{old} = \underbrace{1500}_{\text{I-MEM}} + \underbrace{35\times2}_{\text{Add}} + \underbrace{20\times3}_{\text{MUX}} + \underbrace{100}_{\text{ALU}} + \underbrace{200}_{\text{Regs}} + \underbrace{2000}_{\text{D-MEM}} + \underbrace{500}_{\text{Control}} = 4430\ ps$

$Cost_{new} = \underbrace{1500}_{\text{I-MEM}} + \underbrace{35\times2}_{\text{Add}} + \underbrace{20\times3}_{\text{MUX}} + \underbrace{680}_{\text{ALU}} + \underbrace{200}_{\text{Regs}} + \underbrace{2000}_{\text{D-MEM}} + \underbrace{500}_{\text{Control}} = 5010\ ps$

$CP = \frac{5010}{4430} \times \frac{1900}{1550} \times 0.9 \cong 1.247$ A: 1.25

5. Assume that there are no pipeline stalls and that the breakdown of executed instructions is as follows

add	addi	not	beq	lw	sw
25%	15%	0%	25%	20%	15%

5.1 In what fraction of all cycles is the data memory used?

$\underbrace{20\%}_{\text{lw}} + \underbrace{15\%}_{\text{sw}} = 35\%$

ANS: 35%

5.2 In what fraction of all cycles is the input of the sign-extend circuit needed? What is this circuit doing in cycles in which its input is not needed?

$\underbrace{15\%}_{\text{addi}} + \underbrace{25\%}_{\text{beq}} + \underbrace{20\%}_{\text{lw}} + \underbrace{15\%}_{\text{sw}} = 75\%$

ANS: 75%

5.3 In what fraction of all cycles is the "MemtoReg Mux" used?

$\underbrace{25\%}_{\text{add}} + \underbrace{15\%}_{\text{addi}} + \underbrace{20\%}_{\text{lw}} = 60\%$

ANS: 60%

5.4 In what fraction of all cycles is the "ALUSrc Mux" used?

$\underbrace{25\%}_{\text{add}} + \underbrace{15\%}_{\text{addi}} + \underbrace{25\%}_{\text{beq}} + \underbrace{20\%}_{\text{lw}} + \underbrace{15\%}_{\text{sw}} = 100\%$

ANS: 100%